AGENT OPERATING METHOD · KIT V1.0

Agentic Engineering Kit

Vibe code the Karpathy way — a validated, agent-agnostic method for directing AI coding agents: spec-first, verify-first, autonomy-dialed, delivery-as-contract. One skill, every tool.

> Built from Andrej Karpathy's 2026 corpus (Sequoia AI Ascent talk, autoresearch, nanochat, the 4-rules CLAUDE.md) and validated 2026-11-05 two ways: 30 days of community evidence (58 items across Reddit, X, Hacker News, dev.to) and a 37-source research-notebook cross-check. 5/6 pillars corpus-supported with exact quotes; 1 partial. Golden rule: outsource the thinking, never the understanding.

NO MCP SERVERS · NO API KEYS · ZERO CONTEXT COST · AGENT SKILLS STANDARD

Works with: Claude Code · Google Antigravity (agy CLI + IDE) · Cursor · ZCode · Codex · Opencode · anything via paste-prompt

github.com/webmasterarbez/agentic-engineering-kit · MIT · Validated 2026-11-05

The method in 60 seconds

You are the creative director. The agent is a brilliant intern with perfect recall and no accountability. Claim the leverage without lowering the bar.

> **1 · Context.** Repo has a short AGENTS.md / CLAUDE.md: architecture map, commands, conventions, never-touch list. The context window is the program — trim ruthlessly, consolidate, never accumulate.

> **2 · Spec.** Co-author SPEC.md (detailed enough to be the docs) before any implementation. Human signs off.

> **3 · Autonomy dial.** Per task: on-the-leash (architecture, security, judgment) · autopilot (scoped goals + checkpoints) · autonomous (isolated env + explicit verifier). Default one lower when unsure.

> **4 · Verify-first.** Build the failing test / eval / metric before the implementation. Loops must be resettable, efficient, rewardable.

> **5 · Deliver as a contract.** What changed & why + proof it works (test output, logs) + what to review + what could break.

> **6 · The loop improves, not the agent.** Corrected the same thing twice? Encode it — rule, hook, or test.

The four rules (every task)

RULE	MEANING
Think before coding	State assumptions first. Surface tradeoffs. Ask when uncertain; show interpretations when ambiguous. Never hide confusion.
Simplicity first	Minimum code that solves the stated problem. No unrequested features, no speculative or single-use abstractions.
Surgical changes	Touch only what the task requires. No drive-by refactors. Match repo style. Clean up your own orphans.
Goal-driven execution	Declarative goals with success criteria, then verify. For bugs: failing test first, then make it pass.

Operating principles

- **Outsource thinking, never understanding.** Delegate syntax and API recall freely. Never improvise data models, identity/auth, security, payments, invariants. (Canonical failure: linking accounts by email instead of persistent IDs.)

- **Verifiability beats specifiability.** "Traditional software automates what you can specify. LLMs automate what you can verify." No verifier, no autonomy.
- **Ghosts, not animals.** The agent is a statistical process with jagged intelligence — no hiding confusion, no performing certainty.
- **Stay supervisable.** "If you cannot be a good supervisor, you will not get good results." Every diff must be explainable to the accountable human.

Source: Karpathy 2026 corpus · community-validated 2026-11-05 (58 evidence items + 37-source notebook cross-check) · agentic-engineering-kit

Usage per tool

One installer, six locations. Run `./install.sh`, restart open sessions, then say `karp-mode` — or invoke the skill by name.

TOOL	SKILL LOCATION	INVOKE
Claude Code	`~/.claude/skills/` (user) · `.claude/skills/` (project)	`/karpathy-agentic-engineering` or "karp-mode". Pair with hooks (p.5).
Antigravity CLI (agy)	`~/.gemini/skills/`, `~/.gemini/antigravity-cli/skills/`	`agy -i "karp-mode"` · session modes: `--mode plan|accept-edits|auto`, `--sandbox`
Antigravity IDE	`~/.gemini/antigravity/skills/` · project `.agents/skills/`	Say "karp-mode" in agent chat. Rules: `AGENTS.md`, `GEMINI.md`, `.agents/rules/*.md`
Cursor	`~/.cursor/skills/` · project `.cursor/skills/` or `.agents/skills/`	`/karpathy-agentic-engineering` in Agent chat, or "karp-mode"
ZCode	`~/.agents/skills/` (standard)	New session + "karp-mode". Guardrails in `~/.zcode/AGENTS.md`
Codex / Opencode / other	`~/.agents/skills/` · project `.agents/skills/`	Standard readers pick it up automatically
Anything else	Paste-prompt (no skills needed)	Paste `skills/.../PASTE-PROMPT.md` before your task

Quick start

```
git clone https://github.com/webmasterarbez/agentic-engineering-kit.git
cd agentic-engineering-kit
./install.sh          # installs into every detected tool
./setup-repo.sh ~/projects/my-app --hooks
```

```
#    ↑ optimized CLAUDE.md/AGENTS.md (detected commands + never-touch list + rules,
#      ~35 lines, two [fill me] sections) + merge-safe guardrail hook
# install.sh flags: --all · --uninstall · --sync     setup-repo.sh: --skill · --force ·
--remove-hooks
```

Per-repo setup (the second half of the method): copy `templates/AGENTS-CORE.md` into each project as `AGENTS.md` (or `CLAUDE.md` / `GEMINI.md`), fill the four bracketed sections, keep it under ~60 lines. The skill drives the session; the context file steers every session in that repo.

Admin — inventory

Kit files

FILE	ROLE
`skills/.../SKILL.md`	The skill — method, rules, workflow, autonomy dial
`skills/.../PASTE-PROMPT.md`	Same method as a universal paste-prompt
`templates/AGENTS-CORE.md`	Repo context-file template (4 rules, trimmed)
`LLM.md`	Self-contained instruction doc for direct LLM consumption
`install.sh`	Installer / uninstaller / sync for all locations
`docs/`	This document (HTML + PDF)

MCP servers — none required

The kit is intentionally MCP-free: every connected MCP server injects its full tool schemas into *every* session. The kit adds zero always-on context cost. Optional tooling used only by the validation procedure:

TOOL	PURPOSE	WHERE CONFIGURED
`notebooklm` (CLI)	Grounded Q&A vs the research notebook	`~/.local/bin/notebooklm` (notebooklm-py)
`last30days` (skill)	30-day community evidence harvest	`~/.agents/skills/last30days`
`codegraph`	Symbol navigation in indexed repos	per-tool MCP config; project-scope big servers
`searx` (CLI)	Local rate-limit-free web search	`~/.local/bin/searx`

Scope rule (hard-won): anything big belongs *project-scoped* (that project's `.mcp.json`), never global. One oversized global server cost 15–20k tokens per session before being scoped down. Add per tool: `agy mcp add` · Claude Code `.mcp.json` · Cursor `~/.cursor/mcp.json` · ZCode `~/.zcode/cli/config.json`.

Hooks — enforcement beyond markdown

Markdown rules guide; hooks enforce. Claude Code example (`~/.claude/settings.json`):

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash(git push --force*)|Bash(rm -rf*)",
      "hooks": [{ "type": "command",
        "command": "echo 'BLOCKED: destructive op needs human approval' && exit 2" }]
    }],
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{ "type": "command", "command": "lint-staged || true" }]
    }]
  }
}
```

agy: `--mode plan|accept-edits|auto` + `--sandbox` set the session leash. ZCode: guardrails in `~/.zcode/AGENTS.md`. Correct-twice ledger: promote repeated corrections into the context file or a hook, then delete the ledger line.

Admin — procedures

Install · update · uninstall · backup

```
./install.sh --all          # install everywhere (creates missing dirs)
./install.sh                # re-sync live copies after repo updates
./install.sh --uninstall    # remove from all locations

tar czf ~/backups/aek-$(date +%F).tgz \
  ~/.agents/skills/karpathy-agentic-engineering \
  ~/.claude/skills/karpathy-agentic-engineering \
  ~/.cursor/skills/karpathy-agentic-engineering \
  ~/.gemini/skills/karpathy-agentic-engineering
```

The git repo is the source of truth and the real backup — restore is a clean clone + `./install.sh`. Never edit live copies; edit the repo and re-sync.

Weekly re-validation — Mondays 09:00 (repeatable)

1. **Community evidence** — run the last30days engine on "Karpathy agentic engineering workflow" with targeting `--x-handle=karpathy --github-user=karpathy --subreddits=ChatGPTCoding,ClaudeCode,ExperiencedDevs,AgentsOfAI,singularity`. Note findings that support / contradict / extend the six pillars.
2. **Corpus cross-check** — `notebooklm use a6b9e81a` then `notebooklm ask "For each finding: does Karpathy's work support it, contradict it, or stay silent? Quote exact words."`
3. **Apply** — edit SKILL.md + PASTE-PROMPT.md; label community-vs-Karpathy provenance in the footer; update the validation date and counts.
4. **Publish** — `./install.sh && git add -A && git commit -m "re-validate YYYY-MM" && git push`

> **Precedence when validation conflicts:** the corpus wins — fix the skill to match what Karpathy actually said. Community consensus without corpus support may be added but must be labeled "community-converged addition" (the risk-tier clause is the precedent).

Porting the rules into a project

- **Automated:** `./setup-repo.sh <repo> --hooks` — generates the context file (auto-named: `.claude/` present → `CLAUDE.md`, else `AGENTS.md`) pre-filled with detected commands, never-touch list, detected stack, and the verbatim rules; installs the guardrail hook merge-safe (existing hooks untouched, removable via `--remove-hooks`); `--skill` embeds the skill in the repo.
- Existing files are never overwritten without `--force` — you get a line-count audit instead (over 60 lines = trim candidate).
- **Manual fallback:** copy `templates/AGENTS-CORE.md` to the repo root as `AGENTS.md` (or `CLAUDE.md` / `GEMINI.md`), fill four sections, stay under ~60 lines.

Troubleshooting

SYMPTOM	FIX
Skill doesn't appear in a tool	Check the exact directory (p.3 table); restart the session to rescan
Loads but never triggers	The description field is the trigger surface — keep "karp-mode" / "agentic engineering" in it
Copies diverge	Re-run `./install.sh`; only edit the repo
Context bloat returns	Audit MCP lists per tool; the kit itself is always zero-cost

Provenance & validation record

What this method is built on, and how it was checked.

Sources of the method

- Andrej Karpathy, *"From Vibe Coding to Agentic Engineering"* with Stephanie Zhan, Sequoia AI Ascent 2026 (talk + transcript + his own bearblog writeup).
- Karpathy's `autoresearch` (program.md vs train.py loops), `nanochat`, and MenuGen (the email/Stripe identity-bug example).
- The 4-rules `CLAUDE.md` distilled from his observations on LLM coding pitfalls.
- February 2026 interviews and coverage (Observer, Business Insider, The New Stack).

Validation 2026-11-05

FINDING	COMMUNITY EVIDENCE	CORPUS CHECK
Own the spec, checks, final call; the loop improves, not the agent	X, high engagement	Supported (quotes)
Autonomy dial: leash / autopilot / autonomous	HN + Reddit workflows	Supported
Verify-first: resettable, efficient, rewardable	practitioner guides	Supported
Deskilling is real; supervisor competence gates results	HN (104-pt thread), Anthropic RCT (52 devs)	Supported ("can't outsource understanding")
Correct-twice → encode as hook/rule/test; short context files	r/ClaudeCode (713-upvote setup)	Supported ("trim ruthlessly")
Delivery = proof + risk tier, not a bare diff	PR-contract practice (Osmani)	Partial — proof supported; risk tiers are a community addition

Key quotes

"You can outsource your thinking, but you can't outsource your understanding."

"Traditional computers automate what you can specify in code. This latest round of LLMs can automate what you can verify."

"Vibe coding raises the floor. Agentic engineering raises the ceiling." · "Don't tell it what to do, give it success criteria and watch it go."

"If you cannot be a good supervisor, you will not get good results." · "Treat your context window as the wiring diagram... trim ruthlessly."